Fidelity Cores and Pivots: Verdict-Preserving Configuration Minimization for Security Service Twins

Author Name(s)
Department, Organization
City, Country
email@organization.org

***Abstract*—A security digital twin tests claims about a production service by running its configuration in isolation, so a twin can only be as right as the configuration it copies. We ask which directives actually decide a security verdict, and what goes wrong when a twin copies only some of them. We introduce *fidelity cores* (1-minimal verdict-preserving directive subsets), *pivots* (directives whose individual removal inverts the verdict) and *guard sets* (keys whose edits require re-emulation), and prove sufficient conditions for exact guard-based synchronization. We instrumented Redis, PostgreSQL, Nginx and OpenSSH, plus 16 pinned builds, over 240 seeded configurations with 14,400 drift edits, 1,092 public configuration files and 426 real commits, in about 60,000 server runs. No core exceeded two directives in any corpus. Yet partial twins were brittle: with 90% of directives mirrored at random, 6.6% of verdicts were wrong, because 39% of pivots fall outside the core. Guard sets learned solely from synthetic data reproduced all 450 tested public verdicts while exporting 24% of Redis and 12% of OpenSSH directives. On real commit histories they reduced re-emulations from 426 to 137 and detected all 19 verdict changes. Two hazards hide behind version numbers. The official Redis 7.0.15 image disables protected mode by default where the Ubuntu build enables it, and a platform default profile misclassified 22% of public sshd configurations even at full synchronization. Even a careful static analyzer still disagreed with the servers on a few subtle corner cases.**

***Index Terms*—digital twin, configuration fidelity, security validation, delta debugging, configuration drift, service emulation, misconfiguration**

I. Introduction

Most of a configuration file is housekeeping: buffer sizes, log formats, keep-alive timers. Whether a service is exposed usually comes down to a handful of lines, sometimes just one. A twin that copies the whole file is faithful, but expensive to build, to keep in sync and to trust with customer data. A twin that copies too little gives answers that are confidently wrong. Our earlier configuration-synchronized twin platform [1] quantified the second failure mode. Twins rendered from default settings reached an F1 score of 0.40 on a declared benchmark, whereas synchronized twins were correct on every conclusive run. That told us *that* configuration matters. It did not tell us *how much* of it a verdict needs, or *which* part.

The answer has direct engineering consequences for twin-based validation services. Every exported directive enlarges the customer data held by a shared control plane. Every configuration change triggers re-emulation when synchronization is keyed on content hashes, and re-emulation is the dominant cost in the pipeline. When a twin reports a weakness, operators need to attribute it to specific lines and to know which lines are preventing exposure. All three needs come down to one question: how does a security verdict depend on individual directives?

```
1  server_tokens off;
2  auth_basic "Restricted";
3  autoindex off;
4  add_header X-Content-Type-Options
5  satisfy any;
6  allow 10.77.0.1;
7  client_max_body_size 8m;
8  deny all;
9  keepalive_timeout 15;
```

φ_1: is `/admin/` reachable?
verdict **yes**; core $= \emptyset$ (Nginx is open by default); pivots: lines 5 and 6.

φ_2: is the version disclosed?
verdict **no**; core $= \{1\}$;
pivot: line 1.

Fig. 1. A configuration from the synthetic corpus, with block structure flattened for space. Shaded lines are *pivots* for φ_1: removing either one causes the twin to report "closed". The ostensibly protective lines 2 and 8 can each be removed without affecting the verdict.

Fig. 1 shows why this is not obvious from reading the file. The Nginx configuration contains a server-wide `deny all` and HTTP basic authentication, both conventionally regarded as protective. The `/admin/` path is nevertheless reachable without credentials, because `satisfy any` allows a matching `allow` rule to substitute for authentication. The lines that decide the outcome are not the ones that look important, and a twin that drops either of them gets the answer wrong.

We study the dependence of verdicts on directives empirically, against unmodified servers, through seven research questions:

RQ1 How does verdict accuracy degrade as a function of the fraction of directives a twin mirrors?
RQ2 What is the size of minimal verdict-preserving subsets, are they unique, and do they contain the individually necessary directives?
RQ3 Can twins be constructed from a learned key set, and does such a set transfer from synthetic to real configurations?
RQ4 Under configuration drift, which policy best decides when to re-emulate?

RQ5 To what extent do server versions, and distinct builds of the same version, alter verdicts?

RQ6 Do the findings generalize to configurations and change histories from public repositories?

RQ7 Can a carefully constructed static analyzer substitute for execution?

This paper makes the following contributions:

- **Formal model.** We define verdict-relative fidelity for service twins in terms of fidelity curves, fidelity cores (sufficiency), pivots (necessity) and key guard sets. We prove that key locality and base silence are sufficient conditions for exact selective synchronization and stale-free key-guarded re-emulation (Section III).
- **Measurement harness.** A harness renders configurations onto unmodified servers and 16 pinned builds and obtains verdicts through wire-level probes issued by stock clients (Section IV).
- **Three corpora.** We assemble 240 seeded configurations with 14,400 drift edits, 1,092 distinct public configuration files, and 216 public file histories comprising 426 commits. Raw public files are not redistributed.
- **Declarative service adapters** that reproduce the hand-written adapters exactly and extend the analysis to Mosquitto, MongoDB and MariaDB without code (Section VI).
- **Empirical findings** for all seven questions (Section V), distilled into **design rules** for twin platforms (Section VII).

The results pull in two directions, and that tension is the story. Verdicts are *concentrated*: no core in any corpus exceeded two directives. Verdicts are also *brittle*: mirroring 90% of a configuration uniformly at random still produced a wrong verdict in one case in fifteen. The two fit together once you see that the lines keeping a service safe are often not the lines that make it unsafe. A minimal explanation of a verdict says little about what will break it.

II. Background and Related Work

A. Digital twins and fidelity

The digital twin, a virtual counterpart continuously synchronized with a physical system, originated in product lifecycle management [2] and was subsequently adopted in manufacturing [3]. Security applications emerged for cyber-physical systems [4], [5]. Dietz and Pernul demonstrated simulation, replication and analysis of industrial control systems for security purposes [6], and Holmes *et al.* examined twins both as a defensive capability and as an additional attack surface [7]. A recent survey systematizes twin applications across security operations [8]. The National Academies recommend that a twin be engineered to the fidelity that makes it *fit for purpose* rather than to maximal fidelity [9]. We operationalize this principle: since the purpose of a security twin is a verdict, fidelity is measured per verdict.

B. Execution of real software

CrystalNet emulates production networks by executing real device firmware against production configurations [10], and its authors observe that complete fidelity would amount to replicating production. Cyber ranges make comparable trade-offs for training and testing [11]. Our harness applies the same principle of real software with real configuration at the granularity of individual services, and asks how little configuration suffices.

C. Configuration analysis without execution

Batfish derives a network data plane from router configurations and verifies forwarding properties [12]. Minesweeper encodes control-plane semantics as SMT constraints [13]. Config2Spec infers the policies implied by a network configuration [14]. Attack-graph generators express vulnerability interactions as logical rules over configuration facts [15], [16]. These approaches are efficient and explainable, but their soundness is bounded by the fidelity of their semantic model of each system. **RQ7** quantifies the divergence between a careful model of four services and their observed behavior.

D. Configuration errors, checking and testing

Misconfiguration is a leading cause of production failures [17], and a large share of configuration parameters are rarely modified [18]. PeerPressure localizes faulty settings by statistical comparison against peer machines [19]. EnCore learns correlations among parameters and between parameters and the environment [20]. PCheck synthesizes checkers that emulate deferred uses of configuration values at startup [21]. Ctest binds configuration changes to the software tests that exercise the affected parameters, enabling in-context testing before deployment [22]. Ctest is the closest analogue of our drift guard, but it targets functional failures rather than security verdicts. ConfErr injects human-like configuration errors [23], and Rabkin and Katz precompute the set of options that can explain an observed error [24]. Work on feature interactions shows that options frequently matter only in combination [25]; a fidelity core of size greater than one captures precisely such an interaction. Practitioner surveys [26], studies of infrastructure-as-code security smells [27] and reports on industrial configuration management [28] complete the picture.

E. Input minimization

Delta debugging reduces a failure-inducing input to a 1-minimal subset [29], with worst-case cost quadratic in the number of elements [30]. Subsequent work exploits input structure [31] and reduces the cost of the original algorithm [32]. Program slicing retains only the statements relevant to a given value [33]. A fidelity core is the configuration analogue of a slice with respect to a security verdict, and we compute it by delta debugging.

Table I summarizes the gap. Existing tools either execute real software or reason about security properties. None determines which subset of a configuration a security verdict depends on, or exploits that dependence to decide what to replicate and when to re-execute.

TABLE I
POSITIONING AGAINST REPRESENTATIVE APPROACHES

Approach	Runs real software	Security verdict	Minimal subset	Change handling
Batfish, Minesweeper [12], [13]	no	yes	no	what-if
PeerPressure, EnCore [19], [20]	no	partly	no	no
Ctest [22]	tests	no	no	yes
CrystalNet [10]	yes	no	no	yes
Delta debugging [29]	yes	no	yes	no
Twin platform [1]	yes	yes	no	hash
This work	yes	yes	yes	key guard

III. FORMAL MODEL

A. Configurations, twins and verdicts

A *configuration* $C = \langle d_1, \ldots, d_n \rangle$ is an ordered sequence of directives $d = (\sigma, k, x)$, each comprising a section σ (an Nginx block, an sshd `Match` block, or a pg_hba file), a key k and a value x. Order carries semantics. pg_hba applies the first matching rule; sshd retains the first value obtained for a key; Redis and `postgresql.conf` retain the last. A *base profile* B supplies values for every section–key slot left unassigned by the twin. The *vendor* profile V is empty, so unassigned slots resolve to compiled-in server defaults. The *platform* profile P reproduces the default-twin settings of our earlier benchmark [1]: Redis protected mode enabled, SSH password authentication disabled, PostgreSQL listening on all addresses with a SCRAM catch-all rule, and Nginx version tokens enabled.

For a subset $S \subseteq C$ with order preserved, the twin $T_B(S)$ renders S followed by the entries of B whose slots S leaves unassigned. A *property* φ is a Boolean predicate evaluated over the wire, for example "an unauthenticated remote client can execute `CONFIG GET`". Let $v_\varphi(\cdot)$ denote the verdict returned by a real server and $v^\star = v_\varphi(T_\mathsf{V}(C))$ the source verdict. A server that fails to start yields the verdict *error*, which never equals $v^\star$.

B. Fidelity curves, cores and pivots

Definition 1 (Fidelity curve)**.** *For base B and $p \in [0,1]$, $\varepsilon_B(p)$ is the probability that $v_\varphi(T_B(S)) \neq v^\star$, where $S \subseteq C$ is drawn uniformly at random with $|S| = \mathrm{round}(pn)$.*

Definition 2 (Fidelity core)**.** *$K \subseteq C$ is a* fidelity core *for (C, B, φ) if $v_\varphi(T_B(K)) = v^\star$ and $v_\varphi(T_B(K \setminus \{d\})) \neq v^\star$ for every $d \in K$.*

Definition 3 (Pivot)**.** *$d \in C$ is a* pivot *if $v_\varphi(T_\mathsf{V}(C \setminus \{d\})) \neq v^\star$.*

A core characterizes *sufficiency*: which directives reproduce the verdict. A pivot characterizes *necessity*: which directives the verdict depends on in the source. In Fig. 1 the core for φ_1 is empty, whereas lines 5 and 6 are pivots.

Remark 1 (Cores are not upward-closed)**.** *Consider Redis with `protected-mode no` followed by `requirepass`. The source is not exposed, nor is the empty twin, so $\emptyset$ is a core. Its superset {`protected-mode no`} is exposed. Mirroring a core together with additional directives therefore provides no guarantee, which explains why partial twins can be brittle even when cores are small.*

Remark 2 (Cores are sufficient but not causal)**.** *A core may contain a directive that is inert in the source. In one sshd configuration from our corpus, `PasswordAuthentication no` follows an earlier `yes` and is ignored by the server, yet rendered in isolation it constitutes a valid minimum core. Cores describe what a twin requires; pivots describe what the source depends on.*

Remark 3 (Query complexity)**.** *Treating verdict evaluation as an oracle, ddmin returns a 1-minimal core within $O(n^2)$ queries, and within a logarithmic number when a single directive is decisive [30]. A* minimum *core may require exhaustive search, i.e. $\sum_{i \le k} \binom{n}{i}$ queries for size k. Pivot identification requires exactly n queries.*

C. Guard sets, locality and base silence

A *guard set* G_φ is a set of keys that parameterizes two policies. *Selective synchronization* exports only $C|_G$, the directives whose key belongs to G. *Key-guarded re-emulation* re-executes the twin after an edit only if the edit inserts, deletes, modifies or reorders a directive whose key belongs to G.

Definition 4 (Key locality)**.** *φ is G-*local *over a family of configurations if $C|_G = C'|_G$ implies $v_\varphi(C) = v_\varphi(C')$.*

Proposition 1. *If φ is G-local, then (i) $T_\mathsf{V}(C|_G)$ yields the source verdict, and (ii) key-guarded re-emulation never reports a stale verdict.*

Proof. (i) $T_\mathsf{V}(C|_G)$ renders exactly $C|_G$, whose restriction to G is itself. (ii) An edit touching no key in G leaves $C|_G$ invariant, including the relative order of its directives, since reordering a non-G directive does not permute G directives. By locality the verdict is invariant, so omitting re-emulation is sound. Every other edit triggers re-emulation. □

In practice a platform renders onto a non-empty base, whose defaults may conflict with slots the source leaves unassigned.

Definition 5 (Base silence)**.** *A base B is G-*silent *if it contains no entry whose key belongs to G.*

Proposition 2. *If φ is G-local and B is G-silent, then $T_B(C|_G)$ yields the source verdict for every C in the family.*

Proof. $T_B(C|_G)$ consists of $C|_G$ together with entries of B, none of whose keys belongs to G. Its restriction to G is therefore $C|_G$, and locality implies $v_\varphi(T_B(C|_G)) = v_\varphi(C)$. □

Any base can be made G-silent if the agent reports *absence*. For each guard key unassigned in the source, the twin suppresses the base value and defers to the server default. The platform profile P is not silent with respect to our guard sets; **RQ3** and **RQ6** quantify the consequence.

Algorithm 1 computes cores and pivots. A guard set is then the union of the core and pivot keys seen on training configurations, optionally with an expert list as a floor; the

Algorithm 1 Fidelity core and pivots

Input: configuration C, base B, property φ, server oracle v_φ
Output: core K (or CONFLICT), pivot set Π

$v^\star \leftarrow v_\varphi(T_\forall(C))$
if $v_\varphi(T_B(C)) \neq v^\star$ **then return** CONFLICT
end if
$ok(S) \leftarrow [\, v_\varphi(T_B(S)) = v^\star \,]$ ▷ memoized
if $ok(\emptyset)$ **then** $K \leftarrow \emptyset$
else $K \leftarrow \text{DDMIN}(ok, C)$
end if
$\Pi \leftarrow \{\, d \in C : v_\varphi(T_\forall(C \setminus \{d\})) \neq v^\star \,\}$
return K, Π

agent exports $C|_G$ with an *unset* marker for every absent guard key, and re-emulates only after edits that touch G.

IV. Experimental Methodology

A. Servers, scaffolding and probes

Each evaluation renders the configuration, instantiates a fresh server process, awaits readiness, issues the probe and terminates the process. Ports, PID files, data directories, host keys and the document tree constitute fixed per-slot scaffolding and are therefore excluded as fidelity variables. Server and client communicate over a non-loopback address, since Redis protected mode discriminates loopback clients. The five properties are:

- **Redis**: `redis-cli CONFIG GET maxmemory` succeeds without credentials. Because this is an administrative command, ACL categories and command renaming affect the outcome.
- **PostgreSQL**: `psql -w` as role `synthetic` executes `SELECT 1` without a password.
- **Nginx (two)**: the `Server` header returned for `GET /` discloses a version; `GET /admin/` returns status 200.
- **OpenSSH**: user `synthetic` authenticates with a synthetic password and executes a command.

Probes classify only recognized outcomes, such as `NOAUTH`, "no pg_hba.conf entry" or "Permission denied". Unrecognized outcomes are retried once and otherwise labeled inconclusive; none remained inconclusive after retry. The reference servers were the Ubuntu 24.04 builds of Redis 7.0.15, PostgreSQL 16.15, Nginx 1.24.0 and OpenSSH 9.6p1, deployed in a 4-vCPU, 16 GB container with four parallel worker slots. A verdict cache keyed on the canonical twin eliminated redundant executions. Re-executing 1,000 cached twins on fresh processes reproduced all 1,273 verdicts. A native evaluation (start, probe, stop) took a median of 26–161 ms depending on the service, about 60,000 in total.

B. Pinned builds

For **RQ5**, the same renderers and probes drive 16 pinned container builds attached to the host network namespace, so that each binds exactly the addresses of a native server. The builds comprise Redis 6.0, 6.2, 7.0, 7.2 and 7.4 (official images); PostgreSQL 12, 14, 16 and 17; Nginx 1.20, 1.24 and 1.27; and OpenSSH 8.8p1, 9.3p2, 9.7p1 and 10.0p2.

C. Synthetic corpus and drift model

No public corpus pairs service configurations with wire-verified verdicts, so we first generated one from a seeded catalog. For each service family the catalog specifies legal sections, plausible values and a class label, *security* or *operational*. Each configuration combines 3–10 operational directives with a *posture*, a template modeled on a recurring deployment pattern. Examples include protected mode disabled for a container network, a pg_hba trust rule for a CI subnet, a server-wide `deny all` overridden in one location, and an appended sshd override that never takes effect. A candidate is admitted as a source only if the real server starts and answers conclusively; 3 of 360 candidates were rejected. The first 60 admitted configurations per family (240 in total, 5–16 directives each) form the corpus, partitioned by index parity into training and held-out halves. Class labels are used exclusively for the expert guard set and the drift mix, never for computing cores or pivots.

Each held-out configuration is subjected to three 40-edit traces in which a fraction $q \in \{0.1, 0.3, 0.5\}$ of edits target security keys. An edit modifies a directive with probability 1/2, or inserts, deletes or reorders one with probability 1/6 each. Edits rejected by the server are rolled back, mirroring operator behavior; this occurred 32 times.

D. Public corpus and histories

For **RQ6** we collected `redis.conf`, `sshd_config` and `pg_hba.conf` files from public repositories through GitHub code search, retrieving 2,363 files. A fixed normalization pipeline mapped each file onto the directive model:

- Templates and manual pages (468 files) were rejected, identified by template markers and by the fraction of keys recognized by the server.
- Environment-bound directives that a twin must supply itself were removed: ports, sockets, file paths, includes, modules, TLS material, PAM settings and replication peers.
- Concrete LAN bind addresses were remapped to the twin's address, and every secret was replaced with the synthetic lab password.
- PostgreSQL was studied through pg_hba.conf, with the server listening on all addresses.

Deduplication by normalized content yielded 1,092 distinct configurations: 420 Redis, 339 PostgreSQL and 333 OpenSSH. For 1,500 sampled files we reconstructed the git history from blob-less clones; 216 files had at least two parseable versions, yielding 698 versions and 426 consecutive-version commits. Nginx was excluded because real Nginx configurations depend on site-specific includes and upstreams that our scaffold cannot faithfully reproduce.

E. Static analyzers

For **RQ7** we implemented two analyzers from documentation prior to any comparison and did not tune them on the synthetic results. The *naive* analyzer replicates the original platform's static baseline by reading the last value of a small set of keys. The *careful* analyzer encodes documented semantics: last-wins and first-wins keys, pg_hba first-match evaluation, Nginx per-level inheritance of access rules, sshd `Match` precedence and Redis ACL tokens. To process public files it was extended once, before evaluation, to the full pg_hba column syntax (netmask columns, keyword addresses and comma-separated lists).

F. Metrics

Error rates are averaged per configuration and then across configurations. 95% confidence intervals are obtained by a cluster bootstrap over configurations with 2,000 resamples. For drift we report re-emulations per 100 edits, the fraction of stale steps (twin verdict $\neq$ source verdict) and *late flips*, i.e. true verdict changes not detected at the edit that caused them.

V. Results

A. *RQ1: Partial fidelity is brittle*

Error falls roughly in a straight line as more directives are copied (Fig. 2); there is no point where copying "most" of a file is good enough. Under V, aggregate synthetic error was 45.3% with no directives mirrored, 28.3% at one half (95% CI 25.2–31.5%), 11.6% at 80% and 6.6% (5.5–7.8%) at 90%. SSH exhibits the highest initial error (71.7%), because the vendor default permits password authentication while most sources restrict it in some way. PostgreSQL error remains near 23% until most rules are present, since a trust rule takes effect only in conjunction with `listen_addresses` and all preceding matching rules. Public configurations exhibit the same profile (Section V-F).

Which way the twin errs depends on the base profile. Across partial levels under V, false alarms exceeded false-safe verdicts by 3,658 to 1,980, because random truncation tends to remove a defensive directive while retaining the exposure. The platform profile P halves SSH error at low fidelity but never converges to zero. At full synchronization it still misclassified 1.3% of synthetic verdicts, all for SSH, because sources relying on the vendor default for `PasswordAuthentication` were overridden by the platform value "no". In 88 runs this value additionally prevented sshd from starting, as it contradicted an `AuthenticationMethods` directive in the source. A base profile is not a neutral filler: it is configuration the source never had.

B. *RQ2: Cores are small and mostly unique but exclude pivots*

Across all 300 synthetic configuration–property pairs under V, the maximum core size was two, the median zero and the mean 0.50, approximately 5.1% of a configuration. Cores of size two occurred only for PostgreSQL exposures, which require both `listen_addresses` and the first matching trust rule. Exhaustive enumeration of all subsets of up to two directives confirmed that the ddmin core was a *minimum* core in all 300 pairs under V and all 296 under P. The minimum core was unique in 266 pairs (89%). The remaining 34 admitted two or three minimum cores, 20 of them for SSH, where authentication is frequently blocked by several independent mechanisms. Core computation required 3.2–5.1 server runs on average on the synthetic corpus (at most 22) and 2.6–5.6 on public files (at most 24, for files of up to 87 directives).

TABLE II
Cores (sufficiency) versus pivots (necessity), base V

Corpus	φ	Configs	$\overline{n}$	$\overline{\lvert K\rvert}$	max $\lvert K\rvert$	Pivots	in K
Synthetic	Redis	60	9.0	0.33	1	43	20
	PostgreSQL	60	10.8	0.47	2	33	26
	Nginx leak	60	10.4	0.50	1	27	27
	Nginx admin	60	10.4	0.48	1	62	22
	SSH	60	10.2	0.72	1	29	23
Public	Redis	150	21.4	0.17	1	36	26
	PostgreSQL	150	6.7	0.21	2	33	32
	SSH	150	13.1	0.69	1	60	58

$\overline{n}$: mean directives per configuration. Pivots: directives whose individual removal inverts the verdict; "in K": pivots contained in the ddmin core. Public: seeded sample of admitted files.

Small cores do not mean only a few lines matter (Table II). Of 194 synthetic pivots, 76 (39%) lie outside the vendor core. For Nginx `/admin/`, only 22 of 62 pivots belong to the core, reproducing the pattern of Fig. 1. Conversely, 79–91% of directives labeled *security* belong to no vendor core, including `PermitRootLogin`, `PubkeyAuthentication` and pg_hba rules shadowed by an earlier match. A security label is a statement about a key; a core is a statement about one verdict. Public Redis files are more than twice as long as their synthetic counterparts and public sshd files moderately longer, yet their cores are equally small. In public Redis files, 10 of 36 pivots (`requirepass`, ACL `user` directives and `rename-command`) lie outside the core, each preventing exposure in a server configured with `protected-mode no`.

C. *RQ3: Selective synchronization and cross-corpus transfer*

Table III evaluates twins that receive only directives whose key belongs to a guard set learned on the synthetic training half. When learned from cores alone, G_{core} for Redis reduces to `protected-mode`, and held-out accuracy falls to 91.4%, because cores do not capture the directives that prevent exposure. Adding pivot keys raises accuracy to 99.3% at an export fraction of 24%. The six residual errors involve keys never decisive in the training half, namely Redis `bind` and sshd `DenyUsers`; they are precisely the locality violations anticipated by Proposition 1 for a learned guard. The expert key list was locally complete on this corpus (100%) at an export fraction of 35%.

Applied without modification to public configurations, the guard sets learned from synthetic data transferred without loss. Under V the union guard reproduced all 450 public verdicts while exporting 23.8% of Redis and 12.3% of OpenSSH

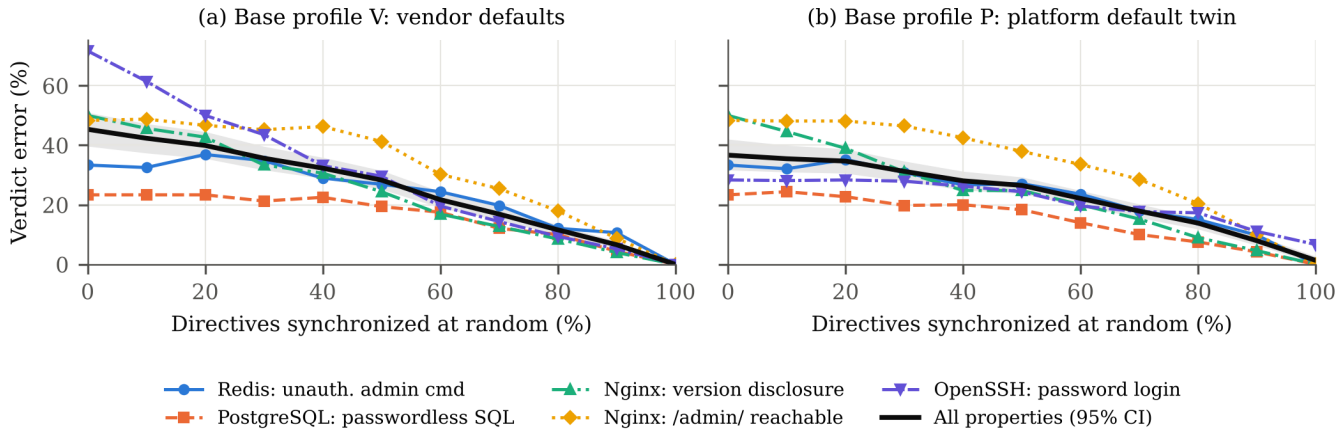


Fig. 2. Fidelity curves on the synthetic corpus: verdict error of twins mirroring a uniformly random fraction of source directives, rendered on (a) vendor defaults and (b) the platform default-twin profile. Black: all properties, with a 95% cluster-bootstrap interval.

TABLE III
SELECTIVE SYNCHRONIZATION WITH LEARNED AND EXPERT GUARD SETS

	Synthetic held-out			Public, transfer (V)		
Guard set	Exp.	Acc. V	Acc. P	Redis	SSH	PG
G_{core}	20.5%	91.4%	86.4%	90.7%	100%	100%
G_{pivot}	23.8%	98.4%	93.7%	–	–	–
$G_{\text{core}} \cup G_{\text{pivot}}$	24.1%	99.3%	94.3%	100%	100%	100%
G_{expert}	34.8%	100%	94.9%	100%	100%	100%
Exported share, public ($G_{\text{core}} \cup G_{\text{pivot}}$)	23.8%	12.3%		100%		

Synthetic: 900 held-out verdicts per row (150 pairs × 6 versions). Public: 150 configurations per family; guard sets learned on the synthetic training half and applied unchanged.

directives. pg_hba files are entirely security-relevant and are therefore exported in full. Cores alone again proved insufficient for Redis (90.7%).

Under P, every synthetic guard set lost approximately five percentage points. The 46 errors of the expert set comprised 26 SSH false-safe verdicts (unassigned `PasswordAuthentication` filled with "no"), 16 PostgreSQL false alarms (unassigned `listen_addresses` filled with "*") and 4 SSH startup failures. On public sshd files the effect was considerably larger: the platform profile misclassified 22% of them even under full synchronization (Section V-F). These are exactly the errors excluded by Proposition 2 once the agent reports absence. With absence markers, P becomes silent for these guard sets and the twin reduces to the V columns by construction.

D. *RQ4: Drift and re-emulation policy*

Re-emulating on every hash change is never stale, but almost all of that work is wasted: at $q = 0.3$ the true verdict changed on only 4.8 of every 100 synthetic edits (Table IV, Fig. 3). Re-emulating on a fixed schedule, the obvious cheap alternative, did badly. Re-emulating every fourth edit cost 25 executions per 100 edits, left 6.35% of steps stale and detected only 93 of 289 flips immediately. The learned union guard

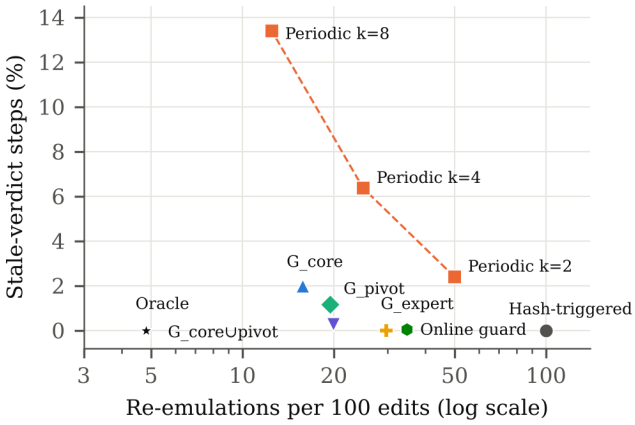


Fig. 3. Cost–staleness trade-off of re-emulation policies over 120 held-out synthetic drift traces of 40 edits ($q = 0.3$). Lower-left is better; the oracle re-emulates only when the true verdict changes.

required 19.9 executions per 100 edits, a five-fold reduction relative to hash-triggered re-emulation, at 0.28% stale steps. Its four late flips were all caused by `bind` or `DenyUsers` edits. The expert guard was never stale, at 29.7 executions per 100 edits. Guards also specialize per property: for Nginx version disclosure the learned guard reduces to `server_tokens` and re-emulated on 6.2 of 100 edits. Periodic audits and online guard extension reduced residual staleness to 0.13% and 0.03% respectively, at a cost comparable to the expert list. The ordering was stable for $q = 0.1$ and $q = 0.5$. The union guard required 6.5 and 32.2 executions per 100 edits (1.05% and 0.58% stale), and the expert guard 9.9 and 49.6 (never stale).

Real histories resemble a low-q trace with coarser edits. Of 426 public commits, 180 modified only comments or environment-bound directives, 246 modified the normalized configuration, and 19 changed a verdict (7 Redis, 7 OpenSSH and 5 PostgreSQL). Per-commit re-emulation would have

TABLE IV
RE-EMULATION POLICIES (SYNTHETIC, $q = 0.3$; PUBLIC HISTORIES)

Policy	Re-emul./100	Stale steps	Late flips
Synthetic: 4,800 edits, 289 true flips			
Hash-triggered (current)	99.8	0.00%	0
Periodic, $k = 2$	50.0	2.38%	143
Periodic, $k = 4$	25.0	6.35%	196
Periodic, $k = 8$	12.5	13.38%	221
Guard G_{core}	15.8	2.00%	24
Guard G_{pivot}	19.5	1.15%	14
Guard $G_{\text{core}} \cup G_{\text{pivot}}$	19.9	0.28%	4
Guard G_{expert}	29.7	0.00%	0
Union guard + audit every 8	29.8	0.13%	4
Online-grown guard	34.8	0.03%	1
Oracle (lower bound)	4.8	0.00%	0
Public histories: 426 commits, 19 true flips			
Per commit	100	0.00%	0
Per directive-hash change	57.7	0.00%	0
Guard $G_{\text{core}} \cup G_{\text{pivot}}$	32.2	0.00%	0
Guard G_{expert}	37.3	0.00%	0
Oracle (lower bound)	4.5	0.00%	0

required 426 executions, and directive-hash re-emulation 246. The learned union guard required 137 and the expert guard 159, and both detected all 19 flips at the commit that introduced them.

E. *RQ5: Version and build sensitivity*

Table V replays 120 configurations per family (60 corpus configurations and 60 drift versions) on every pinned build. For three of the four families, verdicts were invariant. PostgreSQL 12–17, Nginx 1.20–1.27 and OpenSSH 8.8–10.0 returned the reference verdict for every configuration and property, and their core keys were identical across builds (Redis: 56 of 60). For these properties, the version number is not a fidelity variable.

Redis exhibited three distinct classes of version and build hazard:

- **Syntactic.** Redis 6.0 failed to start on 26 configurations using syntax introduced later: 24 ACL directives with the `&*` channel selector and two `bind * -::*` wildcards.
- **Semantic.** Eight configurations changed from exposed on 6.2 to not exposed on 7.0. Prior to 7.0, an explicit `bind` disabled protected mode; from 7.0 onward it does not.
- **Build defaults.** The official 7.0.15 container image disagreed with the Ubuntu 7.0.15 package on 11 configurations, all of which leave `protected-mode` unassigned. Querying each image with an empty configuration identified the cause: every official Redis image tested (6.0–7.4) reports `protected-mode no` by default, whereas the Ubuntu build reports `yes`.

So a twin pinned to "Redis 7.0.15" can give opposite answers depending on where its binary came from. Matching the version is necessary, but it is not enough.

F. *RQ6: Public configurations*

Admission. A source must execute before any twin can be constructed. The reference servers rejected 45 of 420 public Redis files (10.7%), predominantly because of directives specific to other Redis versions. They rejected 43 of 333 sshd files (12.9%) for four reasons: invalid option combinations, options newer than OpenSSH 9.6 (such as post-quantum key-exchange identifiers), seven configurations that never reached readiness, and six requiring keyboard-interactive methods that our PAM-less twin cannot provide. They rejected 10 of 339 pg_hba files (3%). Such rejections are fidelity gaps in their own right, since a twin pinned to the wrong release cannot even start.

TABLE V
VERDICT DIFFERENCES ACROSS PINNED BUILDS (120 CONFIGS PER FAMILY)

Build	Start err.	Differs	Cause
Redis 6.0 (image)	26	40	syntax, bind rule, default
Redis 6.2 (image)	0	19	bind rule, build default
Redis 7.0/7.2/7.4 (image)	0	11 each	build default
PostgreSQL 12/14/16/17	0	0	–
Nginx 1.20/1.24/1.27	0	0	–
OpenSSH 8.8/9.3/9.7/10.0	0	0	–

Reference: Ubuntu 24.04 builds (Redis 7.0.15, PostgreSQL 16.15, Nginx 1.24.0, OpenSSH 9.6p1). "Differs": configurations whose verdict differs from the reference on any property.

Verdicts. Among admitted files, 72 of 375 Redis configurations (19%) accepted unauthenticated administrative commands from a non-loopback client under the reference build's defaults. 34 of 329 pg_hba files (10%) permitted an unlisted role to execute SQL without a password from the twin's address, and 95 of 290 sshd files (33%) permitted password authentication for an ordinary account. These figures characterize the twin setting, not the deployed systems, whose network exposure is unknown.

Curves and cores. Under V, pooled error was 32.2% with no directives mirrored, 14.1% at one half and 3.4% at 90%. Public Redis and PostgreSQL files start lower than the synthetic corpus, since most are secure by default and seldom override it. Public sshd files start high and decline steeply, consistent with the synthetic data. Under P the sshd curve does not converge to zero: 33 of 150 sampled sshd files (22%) admitted no core, because the platform value `PasswordAuthentication no` contradicts files that rely on the vendor default. Cores remained at most two directives in every family (Table II).

G. *RQ7: Static analysis versus execution*

If only one or two lines matter, why not just read them? On the 13,606 distinct synthetic configurations, the naive analyzer achieved only 72–93% accuracy (Table VI). It disregarded sshd's first-value semantics, pg_hba ordering and scoping, and the Nginx rule that any access directive at an inner level replaces the parent's entire rule list. The careful analyzer agreed with the servers on 17,018 of 17,032 synthetic verdicts. Each of its 14 errors traces to documented behavior that is easily misread:

TABLE VI
STATIC ANALYZERS VERSUS REAL-SERVER VERDICTS

	Synthetic, naive		Synthetic, careful		Public
φ	Acc.	FP / FN	Acc.	FP / FN	careful
Redis unauth. admin	88.4%	392 / 4	99.82%	2 / 4	99.2%
PG passwordless SQL	74.5%	868 / 0	100%	0 / 0	100%
Nginx version leak	92.7%	157 / 94	100%	0 / 0	–
Nginx /admin/ open	76.1%	127 / 691	99.77%	8 / 0	–
SSH password login	72.3%	843 / 90	100%	0 / 0	99.0%

Synthetic: 13,606 distinct configurations (corpus and drift versions). Public: 375 Redis, 329 PostgreSQL and 290 OpenSSH admitted files.

- **Redis ACL directives redefine the user.** The directive `user default on nopass ~* &* +@read +@write` produced a user unable to execute `CONFIG`. The analyzer had treated the rules as increments to the implicit `+@all`.
- **ACL directives override `requirepass` irrespective of order.** With `user default ... nopass` preceding `requirepass`, the server still accepted unauthenticated clients. These four errors were missed exposures.
- **`satisfy any` requires an explicit allow.** When no access rule matched the client, the analyzer assumed that the implicit allow satisfied the access condition. Nginx returned 401.

On public files the careful analyzer reached 99.0–100% accuracy (Table VI), but its six errors were of a different nature. Three Redis files defined additional ACL users, and the analyzer, developed against the synthetic catalog, applied every `user` directive to the default user. Two sshd files used syntax outside its model: a wildcard in `AllowGroups` and space-separated alternatives in `AuthenticationMethods`. One sshd disagreement was an artifact of our probe, as a `ForceCommand` replaced the probe's command after successful authentication. The pattern matters more than the count: a static model has to be extended for every bit of syntax it meets in the wild, while a twin simply uses the server's own parser.

VI. GENERALIZATION: DECLARATIVE SERVICE ADAPTERS

The four families above use hand-written adapters. To test whether the model is service-specific, we replaced adapters with declarative YAML specifications. Each spec declares file templates and line formats (flat key–value, INI with flag lines, nested YAML via dotted keys, brace blocks, rule files, dynamic `Match` blocks), a parser for real files, the runtime (native binary or pinned image), readiness, preparation steps, and wire probes with verdict regexes. Cores, pivots, curves, guards and drift policies run unchanged on any spec.

Equivalence. Specs for Redis, PostgreSQL, Nginx and OpenSSH reproduced all 614 verdicts of the hand-written adapters on the 240 corpus configurations and 240 drift versions.

Three spec-only services. Mosquitto 2.0 (MQTT, anonymous publish), MongoDB 7.0 (unauthenticated `listDatabases`) and MariaDB 11.4 (passwordless remote root) were added without code, each with 40 admitted configurations and 400 drift edits (Table VII). The structural findings carried over: cores never exceeded two directives, between 29% (MongoDB) and 62% (MariaDB) of pivots lay outside the vendor core, and the platform profile conflicted with 4–9 sources per service. Fidelity curves again decayed slowly; under P, 17.5–22.5% of MongoDB and MariaDB verdicts remained wrong at full synchronization. The expert guard reproduced every held-out verdict and cut re-emulation from 93–100 to 28–30 per 100 edits. Learned guards matched it for MongoDB and MariaDB but over-fitted on Mosquitto, where operational keys entered the learned set and the export share rose to 82%, which supports keeping the expert list as a floor. The specs also exposed version semantics: on MariaDB 11.4, `skip-grant-tables` does not imply `skip-networking`, so a remote client logs in as root without a password, and Mosquitto 2.0 denies anonymous clients once any listener is declared unless `allow_anonymous` is set. Writing the specs surfaced two harness pitfalls worth recording for practitioners: YAML 1.1 parsers read `off` as a Boolean where mongod reads a string, and quiet logging settings can hide a server's readiness line.

Change gate on real commits. Replaying the key-guarded gate over the 426 public commits of Section V-F, 10 commits introduced an exposure (5 Redis, 4 PostgreSQL, 1 OpenSSH). The gate would have re-emulated 137 commits and blocked all 10, missing none.

Public files for the spec-only services. We repeated the public-corpus pipeline of Section V-F, driven only by the specs' parse rules, on `mosquitto.conf`, `mongod.conf` and `my.cnf` files from GitHub (414 Mosquitto, 168 MongoDB and 373 MariaDB distinct files). Admission exposed environment and version effects again: 80 of 168 MongoDB files failed, 25 of them because removing the environment-bound `logAppend` invalidates a retained `logRotate: reopen` (a normalization artifact) and others because of removed options such as `net.http`. 160 of 373 MariaDB files failed, mostly on file paths and memory sizes the twin cannot provide. Among admitted files, 182 of 368 Mosquitto configurations (49%) accepted anonymous publishers in the twin setting. Cores never exceeded two directives. Guards learned from synthetic data transferred without loss to MongoDB and MariaDB (100% of 60 sampled verdicts each, exporting 63% and 19% of directives) but not entirely to Mosquitto, where the expert list reached 95% and the learned set 92%. The misses involved keys absent from the catalog, such as a listener switched to the WebSocket `protocol`. MariaDB also revealed a *validity pivot*: in 14 files, removing `character-set-server` left a mismatched `collation-server` and the server refused to start. A pivot can therefore protect a verdict by keeping the configuration valid rather than by any security semantics.

Tooling. The machinery is packaged as a command-line tool. `analyze` reports, for a real file, each verdict with its core, its pivots labeled as enabling or protective, security-labeled lines without effect, and keys left to server defaults,

TABLE VII
SPEC-ONLY SERVICES (40 CONFIGURATIONS, 400 DRIFT EDITS EACH, BASE V)

Service	Exp. max $\|K\|$	Piv. (in K)	Expert	H / E / O
Mosquitto	9	2	56 (24)	100%/37%	93.2/29.8/3.5
MongoDB	17	1	24 (17)	100%/30%	99.5/27.8/6.0
MariaDB	11	1	29 (11)	100%/26%	99.0/27.5/8.5

Expert: held-out accuracy / exported share. H/E/O: re-emulations per 100 edits ($q = 0.3$) for hash-triggered, expert guard and oracle. The expert guard left 0.25% of Mosquitto steps stale (one late flip) and none for the others.

with SARIF output for code-scanning pipelines. `export` emits the guard-key subset with explicit *unset* markers (Proposition 2). `fingerprint` compares the default security behavior of builds; on Redis it flagged 8 of 19 probe configurations that differ between the Ubuntu package and the official 7.0.15 image. `gate` implements the key-guarded change check evaluated above.

VII. DISCUSSION

Design rules for twin platforms. The results yield five concrete rules.

1) *Synchronize by guard keys.* Export only directives whose keys belong to the union of learned and expert guard sets, about one quarter of a Redis file and one eighth of an sshd file in the public corpus. This reduces the customer data held by a multi-tenant control plane at no measured cost in accuracy.
2) *Report absence explicitly.* For each guard key, the agent should transmit an explicit *unset* marker so that platform defaults never substitute for vendor defaults (Proposition 2). This substitution was the largest single error source in the public data.
3) *Pin the build, not only the version.* Record binary provenance and verify compiled defaults by querying the running twin. The Redis images demonstrate that a single version string can carry opposite security defaults.
4) *Guard re-emulation by key, with the expert list as a floor.* Key guards reduced re-emulation three- to five-fold on synthetic drift and by 44% relative to directive hashing on real commits, with no stale verdicts in either setting for the expert guard. Learned keys may refine the list per property, but a learned guard cannot cover keys that were never decisive in its training sample.
5) *Attach cores and pivots to findings.* A core provides a minimal explanation ("this verdict is due to line 7"); the pivots provide a remediation map ("removing line 12 would expose the service"). Both are inexpensive to compute on a live twin.

When does static analysis suffice? For four mature services a careful analyst can encode nearly all relevant semantics, and our analyzer did. Its residual disagreements concentrate where documentation is easily misread (ACL redefinition, `satisfy any`) and where real files use syntax the model has not encountered. Static analysis and execution are complementary. The analyzer is instantaneous and explainable; the twin validates it, and fidelity cores determine what the twin may safely omit.

Concentrated but brittle. The heart of this paper is that tiny cores sit alongside a slowly falling fidelity curve. A uniformly random 90% replica omits the decisive directive in about one case in fifteen on synthetic data and one in thirty on public files. What a verdict needs lives in its pivots, not its core. Fidelity belongs to a verdict, not to a file.

VIII. THREATS TO VALIDITY

Corpora. The synthetic configurations derive from postures we designed. The public corpus mitigates this bias, but it is drawn from repositories that publish configurations, which over-represent container setups, tutorials and templates relative to production fleets. Its rejection and exposure rates should not be interpreted as population estimates, and the observed core sizes characterize both corpora rather than a general law. **Normalization.** Removing environment-bound directives, remapping bind addresses and evaluating pg_hba with `listen_addresses='*'` make public files executable, but they also suppress effects present in deployment, such as TLS-only listeners. **Property scope.** Each property is a single wire-level predicate. Richer properties, such as reachability across proxy chains or role-dependent SQL privileges, would likely yield larger cores and weaker locality. **Single-host emulation.** Twins execute as processes or host-network containers on one machine. Firewall, namespace and multi-service effects are out of scope, and our PAM-less sshd twin cannot model PAM stacks. **Learned guards.** Guards were learned from 30 synthetic configurations per family. Their lossless transfer to 450 public verdicts is encouraging but not a guarantee, which motivates retaining the expert list as a floor. **Analyzer authorship.** The static analyzers were implemented by the authors from documentation, prior to the comparisons. A different implementer would err differently, so the disagreements are reported as instances of semantic corner cases rather than as a ranking of tools.

IX. ETHICS AND DATA HANDLING

All servers ran in an isolated container against synthetic accounts and a synthetic password, and no external system was probed. Public files were obtained through GitHub code search and anonymous reads of public repositories. Raw files were retained outside the released artifact, and secrets were replaced prior to storing any parsed result. The released data identify configurations solely by content hash and contain no repository names, so no reported verdict can be attributed to a specific project. Exposure figures describe the twin setting, not the systems from which the files originate.

X. CONCLUSION

We asked how much configuration a security twin needs, and the answer has two parts. To *reproduce* a verdict, very little: at most two directives in every case measured, synthetic or public. To *keep* reproducing it as files change and platforms

fill in defaults, a twin must track the larger set of directives whose removal would invert it, mark unset keys explicitly, and run the same build as the source, not just the same version. Treating fidelity as a property of verdicts rather than of files allows a twin platform to export a quarter of a configuration and re-emulate only a fraction as often, while agreeing with the real servers on every public verdict we tested. It also explains why a twin built from defaults, or from most of a file, can be confidently wrong. The same conclusions held for three further services added purely through declarative specifications. The harness, service specifications, generator, normalized corpora, raw results and analysis scripts accompany this paper.

References

[1] "Digital Twin: Discovery, emulation and bounded hunting," source code and evaluation artifacts, GitHub repository poojarishreyas/Digital_twin, 2026. [Online]. Available: https://github.com/poojarishreyas/Digital_twin

[2] M. Grieves and J. Vickers, "Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems," in *Transdisciplinary Perspectives on Complex Systems*, F.-J. Kahlen, S. Flumerfelt, and A. Alves, Eds. Cham, Switzerland: Springer, 2017, pp. 85–113.

[3] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, "Digital twin in industry: State-of-the-art," *IEEE Trans. Ind. Informat.*, vol. 15, no. 4, pp. 2405–2415, Apr. 2019.

[4] M. Eckhart and A. Ekelhart, "Towards security-aware virtual environments for digital twins," in *Proc. 4th ACM Workshop Cyber-Physical System Security (CPSS)*, 2018, pp. 61–72.

[5] M. Eckhart and A. Ekelhart, "Digital twins for cyber-physical systems security: State of the art and outlook," in *Security and Quality in Cyber-Physical Systems Engineering*, S. Biffl, M. Eckhart, A. Lüder, and E. Weippl, Eds. Cham, Switzerland: Springer, 2019, pp. 383–412.

[6] M. Dietz and G. Pernul, "Unleashing the digital twin's potential for ICS security," *IEEE Security & Privacy*, vol. 18, no. 4, pp. 20–27, 2020.

[7] D. Holmes, M. Papathanasaki, L. Maglaras, M. A. Ferrag, S. Nepal, and H. Janicke, "Digital twins and cyber security – solution or challenge?" in *Proc. 6th South-East Europe Design Automation, Computer Engineering, Computer Networks and Social Media Conf. (SEEDA-CECNSM)*, 2021, pp. 1–8.

[8] P. Empl, D. Koch, M. Dietz, and G. Pernul, "Digital twins in security operations: State of the art and future perspectives," *ACM Comput. Surv.*, vol. 58, no. 1, Art. no. 18, 2025.

[9] National Academies of Sciences, Engineering, and Medicine, *Foundational Research Gaps and Future Directions for Digital Twins*. Washington, DC, USA: The National Academies Press, 2024.

[10] H. H. Liu *et al.*, "CrystalNet: Faithfully emulating large production networks," in *Proc. 26th ACM Symp. Operating Systems Principles (SOSP)*, 2017, pp. 599–613.

[11] M. M. Yamin, B. Katt, and V. Gkioulos, "Cyber ranges and security testbeds: Scenarios, functions, tools and architecture," *Computers & Security*, vol. 88, Art. no. 101636, Jan. 2020.

[12] A. Fogel, S. Fung, L. Pedrosa, M. Walraed-Sullivan, R. Govindan, R. Mahajan, and T. Millstein, "A general approach to network configuration analysis," in *Proc. 12th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2015, pp. 469–483.

[13] R. Beckett, A. Gupta, R. Mahajan, and D. Walker, "A general approach to network configuration verification," in *Proc. ACM SIGCOMM*, 2017, pp. 155–168.

[14] R. Birkner, D. Drachsler-Cohen, L. Vanbever, and M. Vechev, "Config2Spec: Mining network specifications from network configurations," in *Proc. 17th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2020, pp. 969–984.

[15] X. Ou, S. Govindavajhala, and A. W. Appel, "MulVAL: A logic-based network security analyzer," in *Proc. 14th USENIX Security Symp.*, 2005, pp. 113–128.

[16] O. Sheyner, J. Haines, S. Jha, R. Lippmann, and J. M. Wing, "Automated generation and analysis of attack graphs," in *Proc. IEEE Symp. Security and Privacy*, 2002, pp. 273–284.

[17] Z. Yin, X. Ma, J. Zheng, Y. Zhou, L. N. Bairavasundaram, and S. Pasupathy, "An empirical study on configuration errors in commercial and open source systems," in *Proc. 23rd ACM Symp. Operating Systems Principles (SOSP)*, 2011, pp. 159–172.

[18] T. Xu, L. Jin, X. Fan, Y. Zhou, S. Pasupathy, and R. Talwadker, "Hey, you have given me too many knobs!: Understanding and dealing with over-designed configuration in system software," in *Proc. 10th Joint Meeting European Software Engineering Conf. and ACM SIGSOFT Symp. Foundations of Software Engineering (ESEC/FSE)*, 2015, pp. 307–319.

[19] H. J. Wang, J. C. Platt, Y. Chen, R. Zhang, and Y.-M. Wang, "Automatic misconfiguration troubleshooting with PeerPressure," in *Proc. 6th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2004, pp. 245–258.

[20] J. Zhang, L. Renganarayana, X. Zhang, N. Ge, V. Bala, T. Xu, and Y. Zhou, "EnCore: Exploiting system environment and correlation information for misconfiguration detection," in *Proc. 19th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2014, pp. 687–700.

[21] T. Xu, X. Jin, P. Huang, Y. Zhou, S. Lu, L. Jin, and S. Pasupathy, "Early detection of configuration errors to reduce failure damage," in *Proc. 12th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2016, pp. 619–634.

[22] X. Sun, R. Cheng, J. Chen, E. Ang, O. Legunsen, and T. Xu, "Testing configuration changes in context to prevent production failures," in *Proc. 14th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2020, pp. 735–751.

[23] L. Keller, P. Upadhyaya, and G. Candea, "ConfErr: A tool for assessing resilience to human configuration errors," in *Proc. IEEE/IFIP Int. Conf. Dependable Systems and Networks (DSN)*, 2008, pp. 157–166.

[24] A. Rabkin and R. H. Katz, "Precomputing possible configuration error diagnoses," in *Proc. 26th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2011, pp. 193–202.

[25] N. Siegmund, S. S. Kolesnikov, C. Kästner, S. Apel, D. Batory, M. Rosenmüller, and G. Saake, "Predicting performance via automated feature-interaction detection," in *Proc. 34th Int. Conf. Software Engineering (ICSE)*, 2012, pp. 167–177.

[26] M. Sayagh, N. Kerzazi, B. Adams, and F. Petrillo, "Software configuration engineering in practice: Interviews, survey, and systematic literature review," *IEEE Trans. Softw. Eng.*, vol. 46, no. 6, pp. 646–673, 2020.

[27] A. Rahman, C. Parnin, and L. Williams, "The seven sins: Security smells in infrastructure as code scripts," in *Proc. 41st IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2019, pp. 164–175.

[28] C. Tang *et al.*, "Holistic configuration management at Facebook," in *Proc. 25th ACM Symp. Operating Systems Principles (SOSP)*, 2015, pp. 328–343.

[29] A. Zeller and R. Hildebrandt, "Simplifying and isolating failure-inducing input," *IEEE Trans. Softw. Eng.*, vol. 28, no. 2, pp. 183–200, Feb. 2002.

[30] R. Hildebrandt and A. Zeller, "Simplifying failure-inducing input," in *Proc. ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA)*, 2000, pp. 135–145.

[31] G. Misherghi and Z. Su, "HDD: Hierarchical delta debugging," in *Proc. 28th Int. Conf. Software Engineering (ICSE)*, 2006, pp. 142–151.

[32] R. Hodován and Á. Kiss, "Practical improvements to the minimizing delta debugging algorithm," in *Proc. 11th Int. Joint Conf. Software Technologies (ICSOFT-EA)*, 2016, pp. 241–248.

[33] M. Weiser, "Program slicing," *IEEE Trans. Softw. Eng.*, vol. SE-10, no. 4, pp. 352–357, Jul. 1984.